

4.2.19: Direction and Normal Vectors

Taraka Abhinav - EE25BTECH11016

October 1, 2025

Question

Find the direction and normal vectors of the following line:

$$y = -x + 2$$

Step 1: Standard and Vector Form

First, we rewrite the line equation in the standard form $ax + by = c$.

$$x + y = 2 \quad (1)$$

The vector form of a line is $\mathbf{n}^T \mathbf{x} = c$, where $\mathbf{x} = \begin{pmatrix} x \\ y \end{pmatrix}$ and $\mathbf{n}$ is the normal vector.

Step 2: Finding the Normal Vector

By comparing the standard form $x + y = 2$ with the vector form $\mathbf{n}^T \mathbf{x} = c$, we can directly identify the components of the normal vector's transpose.

$$\mathbf{n}^T = \begin{pmatrix} 1 & 1 \end{pmatrix} \quad (2)$$

Therefore, the normal vector $\mathbf{n}$ is the column vector:

$$\mathbf{n} = \begin{pmatrix} 1 \\ 1 \end{pmatrix} \quad (3)$$

Step 3: Finding the Direction Vector

The direction vector, $\mathbf{m}$, is perpendicular to the normal vector, $\mathbf{n}$. This means their dot product is zero ($\mathbf{n}^T \mathbf{m} = 0$).

For a normal vector $\mathbf{n} = \begin{pmatrix} a \\ b \end{pmatrix}$, a perpendicular vector is $\mathbf{m} = \begin{pmatrix} b \\ -a \end{pmatrix}$.

$$\mathbf{m} = \begin{pmatrix} 1 \\ -1 \end{pmatrix} \quad (4)$$

Verification:

$$\mathbf{n}^T \mathbf{m} = \begin{pmatrix} 1 & 1 \end{pmatrix} \begin{pmatrix} 1 \\ -1 \end{pmatrix} = (1)(1) + (1)(-1) = 0 \quad (5)$$

```
/* This C code is intended to be compiled into a
shared library (.so) for the Python+C example.
Compile with:
gcc -shared -o line_vectors.so -fPIC line_vectors.c
*/
#include <stdio.h>

void get_vectors(double n[2], double m[2]) {
    // Equation:  $x + y = 2$ 
    n[0] = 1.0; // Normal vector
    n[1] = 1.0;

    m[0] = 1.0; // Direction vector
    m[1] = -1.0;
}
```

Python + C Code

```
import ctypes
import numpy as np
import matplotlib.pyplot as plt

# Load shared library
lib = ctypes.CDLL("./line_vectors.so")

# Define C function signature
lib.get_vectors.argtypes = [ctypes.POINTER(ctypes.c_double),
                             ctypes.POINTER(ctypes.c_double)]

def get_vectors():
    n = (ctypes.c_double * 2)()
    m = (ctypes.c_double * 2)()
    lib.get_vectors(n, m)
    return np.array([n[0], n[1]]), np.array([m[0], m[1]])

# Call C function
n, m = get_vectors()
```

Pure Python Code

```
import numpy as np
import matplotlib.pyplot as plt

# Given line:  $y = -x + 2$  or  $x + y = 2$ 
n = np.array([1, 1]) # Normal vector
m = np.array([1, -1]) # Direction vector

print("Normal vector =", n)
print("Direction vector =", m)
```


Python Plotting Code

```
# Plot the line
x_vals = np.linspace(-2, 4, 100)
y_vals = -x_vals + 2
plt.plot(x_vals, y_vals, 'g', label="Line  $y=-x+2$ ")

# Show vectors at point (1,1) on the line
point = np.array([1, 1])
plt.quiver(*point, n[0], n[1], angles="xy",
           scale_units="xy", scale=1, color='r',
           label="Normal Vector n")
plt.quiver(*point, m[0], m[1], angles="xy",
           scale_units="xy", scale=1, color='b',
           label="Direction Vector m")

plt.grid(True); plt.gca().set_aspect('equal')
plt.legend(); plt.title("Line with Vectors")
plt.savefig("figs/Figure_1.png"); plt.show()
```

Plot of the Line and its Vectors

